



# CS-317

# Operating Systems

LECTURE 15





# Concurrency

BOOK 1 CHAPTER 5

# In the last lecture...

- ▶ We explored binary semaphores.
- ▶ We implemented mutual exclusion using semaphores.
- ▶ We implemented synchronization using semaphores.







# Semaphores

CONCEPT, USE AND IMPLEMENTATION

# Implementation of semaphores

- ▶ It is imperative that the `semWait` and `semSignal` operations be implemented as atomic primitives.
- ▶ One obvious way is to implement them in hardware or firmware.
- ▶ Another alternative is to use one of the hardware-supported schemes for mutual exclusion. We shall implement semaphores:
  - ▶ Using compare and swap,
  - ▶ By enabling and disabling interrupts.



# Implementation with compare & swap

- ▶ The semaphore is a structure and includes an integer **flag**.
- ▶ This involves busy waiting.
- ▶ The **semWait** and **semSignal** operations are relatively short, so the amount of busy waiting should be minor.

```
semWait(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
        /* do nothing */;
    s.count--;
    if (s.count < 0) {
        /* place this process in s.queue*/;
        /* block this process (must also set
s.flag to 0) */;
    }
    s.flag = 0;
}
```

```
semSignal(s)
{
    while (compare_and_swap(s.flag, 0 , 1) == 1)
        /* do nothing */;
    s.count++;
    if (s.count <= 0) {
        /* remove a process P from s.queue */;
        /* place process P on ready list */;
    }
    s.flag = 0;
}
```

```
struct semaphore {
    int count;
    queue_t queue;
    int flag;
};
```





```
semWait (s)
{
    inhibit interrupts;
    s.count--;
    if (s.count < 0) {
        /*place this process in s.queue*/;
        /*block this process*/;
    }
    allow interrupts;
}
```

```
semSignal (s)
{
    inhibit interrupts;
    s.count++;
    if (s.count <= 0) {
        /*remove a process P from s.queue*/;
        /*place process P on ready list*/;
    }
    allow interrupts;
}
```

# Implementation with interrupt disabling

- ▶ In a uniprocessor, we inhibit interrupts during **semWait** or **semSignal** operations.
- ▶ The **semWait** and **semSignal** operations are relatively short, so the time interrupts remain disabled is small.

```
struct semaphore {
    int count;
    queue_t queue;
};
```



# Classical Synchronization Problems

## THE PRODUCER / CONSUMER PROBLEM (PCP)





# Problem statement

- ▶ There are one or more **producers** generating some type of data (records, characters) and placing these in a **buffer**.
- ▶ There is a single **consumer** that is taking items out of the buffer one at a time.
- ▶ Ensure that:
  - ▶ Only one agent (producer or consumer) may access the buffer at any one time,
  - ▶ The producer does not add data into a full buffer, and
  - ▶ The consumer does not remove data from an empty buffer.



# Versions of the producer / consumer problem

- ▶ The problem has two versions, determined by the nature of the buffer.
- ▶ The buffer shared by the producers and the consumer may be of a finite size or it may be able to grow to any length.
- ▶ The two versions of the problem are called:
  - ▶ The producer / consumer problem with infinite buffer, and
  - ▶ The producer / consumer problem with finite buffer.



# The Producer / Consumer Problem (PCP)

WITH INFINITE BUFFER

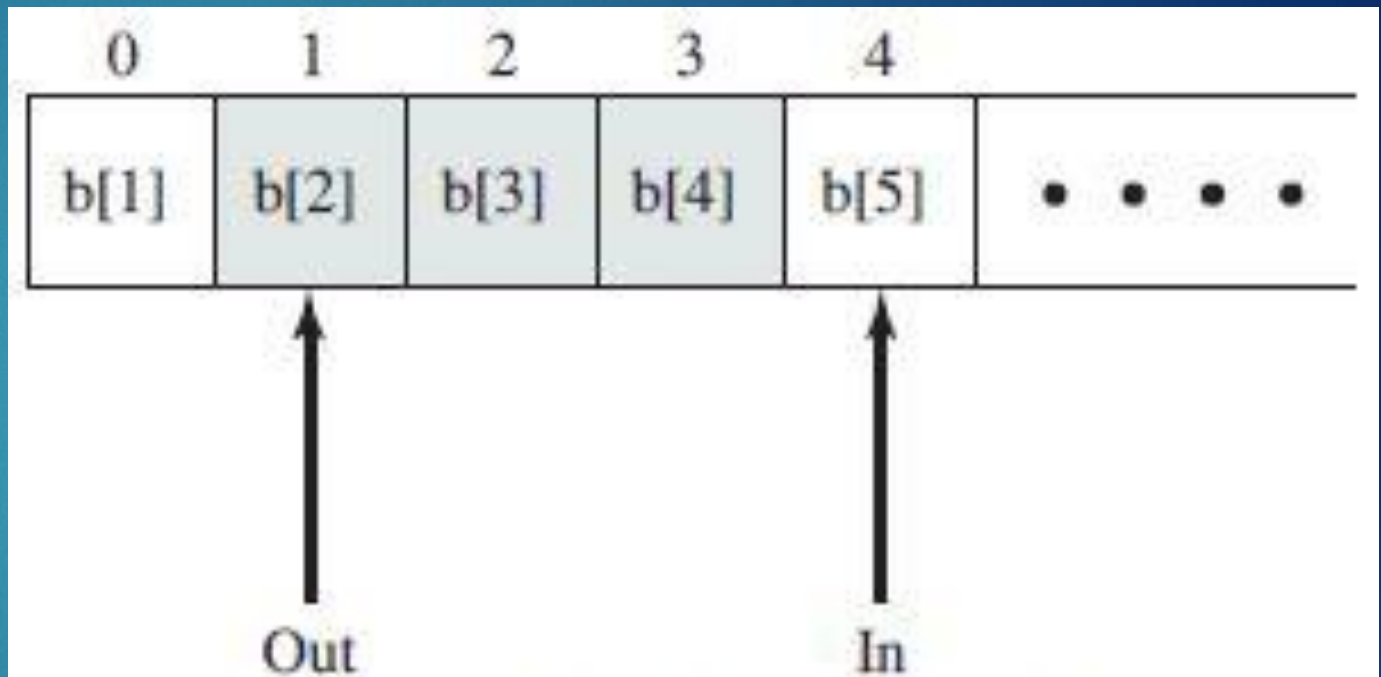




# Basic assumptions and code

```
producer:  
while (true) {  
    /*produce item v*/;  
    b[in] = v;  
    in ++; }
```

```
consumer:  
while (true) {  
    while (in <= out)  
        /*do nothing*/;  
    w = b[out];  
    out ++;  
    /*consume item w*/; }
```



*Note: Shaded area indicates portion of buffer that is occupied*

**Figure 5.8** Infinite Buffer for the Producer/Consumer Problem



```
/* program producerconsumer */  
int n;  
binary_semaphore s = 1, delay = 0;
```

```
void producer()
```

```
{
```

```
    while (true) {
```

```
        produce();
```

```
        semWaitB(s);
```

```
        append();
```

```
        n++;
```

```
        if (n==1) semSignalB(delay)
```

```
        semSignalB(s);
```

```
    }
```

```
}
```

```
void consumer()
```

```
{
```

```
    semWaitB(delay);
```

```
    while (true) {
```

```
        semWaitB(s);
```

```
        take();
```

```
        n--;
```

```
        semSignalB(s);
```

```
        consume();
```

```
        if (n==0) semWaitB(delay);
```

```
    }
```

```
}
```

```
void main()
```

```
{
```

```
    n = 0;
```

```
    parbegin (producer, consumer);
```

```
}
```

Critical section

Critical section

**Figure 5.9** An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores



```

/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}
void consumer()
{
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        semSignalB(s);
        consume();
        if (n==0) semWaitB(delay);
    }
}
void main()
{
    n = 0;
    parbegin (producer, consumer);
}

```

Critical section

**Table 5.4** Possible Scenario for the Program of Figure 5.9

|    | Producer                         | Consumer                       | s | n  | Delay |
|----|----------------------------------|--------------------------------|---|----|-------|
| 1  |                                  |                                | 1 | 0  | 0     |
| 2  | semWaitB(s)                      |                                | 0 | 0  | 0     |
| 3  | n++                              |                                | 0 | 1  | 0     |
| 4  | if (n==1)<br>(semSignalB(delay)) |                                | 0 | 1  | 1     |
| 5  | semSignalB(s)                    |                                | 1 | 1  | 1     |
| 6  |                                  | semWaitB(delay)                | 1 | 1  | 0     |
| 7  |                                  | semWaitB(s)                    | 0 | 1  | 0     |
| 8  |                                  | n--                            | 0 | 0  | 0     |
| 9  |                                  | semSignalB(s)                  | 1 | 0  | 0     |
| 10 | semWaitB(s)                      |                                | 0 | 0  | 0     |
| 11 | n++                              |                                | 0 | 1  | 0     |
| 12 | if (n==1)<br>(semSignalB(delay)) |                                | 0 | 1  | 1     |
| 13 | semSignalB(s)                    |                                | 1 | 1  | 1     |
| 14 |                                  | if (n==0)<br>(semWaitB(delay)) | 1 | 1  | 1     |
| 15 |                                  | semWaitB(s)                    | 0 | 1  | 1     |
| 16 |                                  | n--                            | 0 | 0  | 1     |
| 17 |                                  | semSignalB(s)                  | 1 | 0  | 1     |
| 18 |                                  | if (n==0)<br>(semWaitB(delay)) | 1 | 0  | 0     |
| 19 |                                  | semWaitB(s)                    | 0 | 0  | 0     |
| 20 |                                  | n--                            | 0 | -1 | 0     |
| 21 |                                  | semSignalB(s)                  | 1 | -1 | 0     |

Note: White areas represent the critical section controlled by semaphore s.

**Figure 5.9** An Incorrect Solution to the Infinite-Buffer Problem Using Binary Semaphores



```

/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
    while (true) {
        produce();
        semWaitB(s);
        append();
        n++;
        if (n==1) semSignalB(delay);
        semSignalB(s);
    }
}

void consumer()
{
    int m; /* a local variable */
    semWaitB(delay);
    while (true) {
        semWaitB(s);
        take();
        n--;
        m = n;
        semSignalB(s);
        consume();
        if (m==0) semWaitB(delay);
    }
}

void main()
{
    n = 0;
    parbegin (producer, consumer);
}

```

Critical section

|    | Producer          | Consumer                     | m | n | s | delay |
|----|-------------------|------------------------------|---|---|---|-------|
| 1  |                   |                              |   | 0 | 1 | 0     |
| 2  |                   | Blocks at<br>semWaitB(delay) |   | 0 | 1 | 0     |
| 3  | semWaitB(s)       |                              |   | 0 | 0 | 0     |
| 4  | n++               |                              |   | 1 | 0 | 0     |
| 5  | semSignalB(delay) | unblocks                     |   | 1 | 0 | 0     |
| 6  | semSignalB(s)     |                              |   | 1 | 1 | 0     |
| 7  |                   | semWaitB(s)                  |   | 1 | 0 | 0     |
| 8  |                   | n--                          |   | 0 | 0 | 0     |
| 9  |                   | m=n                          | 0 | 0 | 0 | 0     |
| 10 |                   | semSignalB(s)                | 0 | 0 | 1 | 0     |
| 11 |                   | Blocks at<br>semWaitB(delay) | 0 | 0 | 1 | 0     |
| 12 | semWaitB(s)       |                              |   | 0 | 0 | 0     |
| 13 | n++               |                              |   | 1 | 0 | 0     |
| 14 | semSignalB(delay) | unblocks                     |   | 1 | 0 | 0     |
| 15 | semSignalB(s)     |                              |   | 1 | 1 | 0     |

**Figure 5.10** A Correct Solution to the Infinite-Buffer Problem Using Binary Semaphores

```
/* program producerconsumer */  
semaphore n = 0, s = 1;  
  
void producer()  
{  
    while (true) {  
        produce();  
        semWait(s);  
        append();  
        semSignal(s);  
        semSignal(n);  
    }  
}  
  
void consumer()  
{  
    while (true) {  
        semWait(n);  
        semWait(s);  
        take();  
        semSignal(s);  
        consume();  
    }  
}  
  
void main()  
{  
    parbegin (producer, consumer);  
}
```

Critical section

Critical section

**Figure 5.11** A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores



```

/* program producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(n);
        semSignal(s);
    }
}
void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}

```

What if the semSignals shown in the blue box are reversed?

- There is no effect on the overall code, because the consumer must wait on both semaphores.

**Figure 5.11** A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores





```

/* program producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
    while (true) {
        produce();
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}
void consumer()
{
    while (true) {
        semWait(s);
        semWait(n);
        take();
        semSignal(s);
        consume();
    }
}
void main()
{
    parbegin (producer, consumer);
}

```

What if the semWaits shown in the blue box are reversed?

- ▶ Assume the buffer is empty, n is zero.
- ▶ Assume a consumer arrives. The consumer decrements s to zero and enters critical section. The consumer is blocked on n.
- ▶ Now no producer can enter critical section.
- ▶ This is a deadlock.

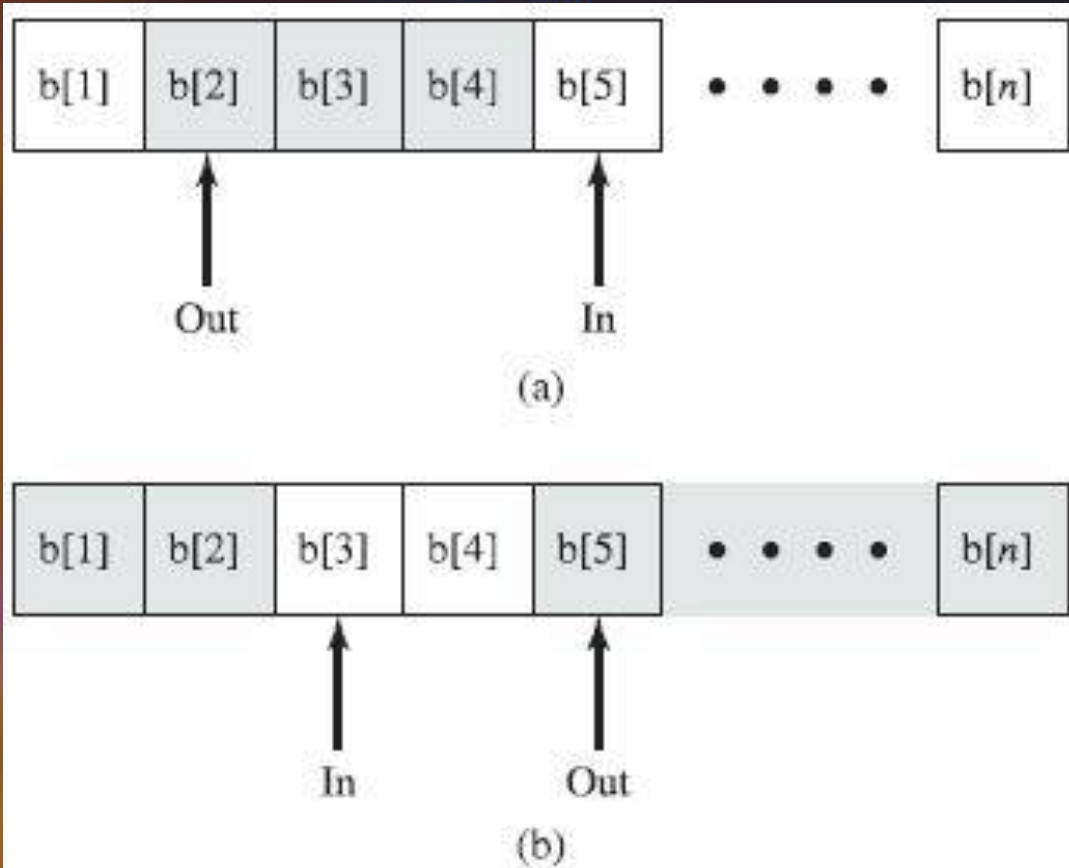
**Figure 5.11** A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores



# The Producer / Consumer Problem (PCP)

WITH FINITE BUFFER





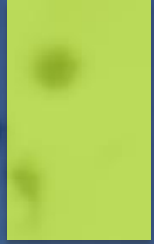
**Figure 5.12** Finite Circular Buffer for the Producer/Consumer Problem

## Finite circular buffer

- ▶ The buffer is treated as a circular storage.
- ▶ Pointer values must be expressed modulo the size of the buffer.
- ▶ Valid data items can be found from the slot pointed to by **Out** to the slot just before the one pointed to by **In**.
  - a) Valid data items:  $b[2]$  to  $b[4]$ , both inclusive.
  - b) Valid data items:  $b[5]$  to  $b[2]$ , both inclusive.





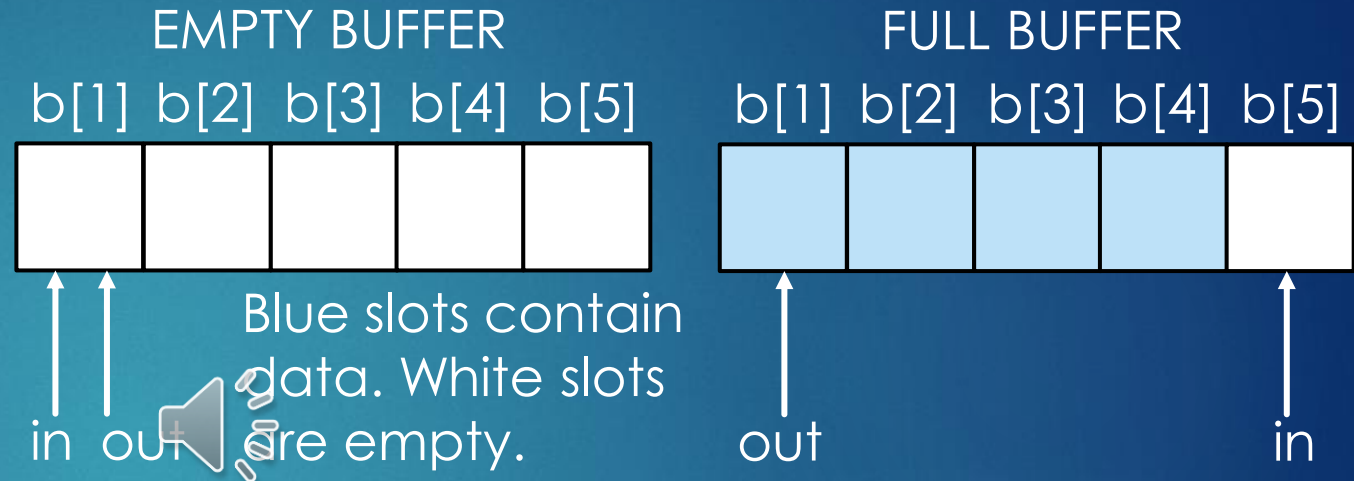


| Do this: | To:      | When:                            |
|----------|----------|----------------------------------|
| Block    | Producer | The buffer is full               |
| Block    | Consumer | The buffer is empty              |
| Unblock  | Producer | Item is removed from the buffer  |
| Unblock  | Consumer | Item is inserted into the buffer |



# Producer and consumer code

- ▶ **in** and **out** are initialized to 1.
- ▶ **n** is the size of the buffer, which is an array **b**. Array index in **b** goes from 1 to **n**.
- ▶ Empty buffer: **in = out**
- ▶ Full buffer: **(in + 1) % n = out**



```
producer:
while (true) {
    /*produce item v*/;
    while ((in + 1) == out)
        /*do nothing*/;
    b[in] = v;
    in = (in + 1) % n;
}
```

```
consumer:
while (true) {
    while (in == out)
        /*do nothing*/;
    w = b[out];
    out = (out + 1) % n;
    /*consume item w*/;
}
```

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n = 0, e = sizeofbuffer;

void producer()
{
    while (true) {
        produce();
        semWait(e);
        semWait(s);
        append();
        semSignal(s);
        semSignal(n);
    }
}

void consumer()
{
    while (true) {
        semWait(n);
        semWait(s);
        take();
        semSignal(s);
        semSignal(e);
        consume();
    }
}

void main()
{
    parbegin (producer, consumer);
}
```

Critical section

Critical section

**Figure 5.13** A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores





# Message Passing

CONCEPT, USE AND IMPLEMENTATION



# The need for message passing

- ▶ When processes interact with one another, two fundamental requirements must be satisfied: synchronization and communication.
- ▶ Processes need to be synchronized to enforce mutual exclusion.
- ▶ Cooperating processes may need to exchange information.
- ▶ Message passing lends itself to implementation in distributed systems as well as in shared-memory multiprocessor and uniprocessor systems.



# Message passing primitives

- ▶ The actual function of message passing is normally provided in the form of a pair of primitives:
  - ▶ **send** (destination, message)
  - ▶ **receive** (source, message)
- ▶ A process sends information in the form of a message to another process designated by a destination.
- ▶ A process receives information by executing the receive primitive, indicating the source and the message.





# Synchronization

- ▶ When a send primitive is executed in a process:
  - ▶ The sending process is blocked until the message is received (**blocking send**), or
  - ▶ The sending process continues to its next line of code (**nonblocking send**).
- ▶ When a process issues a receive primitive:
  - ▶ If a message has previously been sent, the message is received, and execution continues.
  - ▶ If there is no waiting message:
    - ▶ the process is blocked until a message arrives (**blocking receive**), or
    - ▶ the process continues to execute, abandoning the attempt to receive (**nonblocking receive**).





# Send-receive combinations

- ▶ **Blocking send, blocking receive:**

- ▶ Both the sender and receiver are blocked until the message is delivered.
- ▶ This is sometimes referred to as a rendezvous.
- ▶ This combination allows for tight synchronization between processes.

- ▶ **Nonblocking send, blocking receive:**

- ▶ Although the sender may continue, the receiver is blocked until the requested message arrives.
- ▶ It allows a process to send one or more messages to a variety of destinations as quickly as possible.

- ▶ **Nonblocking send, nonblocking receive:**

- ▶ Neither party is required to wait.



# Acknowledgement

- ▶ The blocking send waits for the receiver to signal that it has received a message and unblocks on this signal.
- ▶ The blocking receive waits for the message to arrive in a buffer and unblocks on its arrival.
- ▶ Can you tell why the combination **blocking send, nonblocking receive** is never used?
  - ▶ The sender would block for a signal from the receiver, but if the receiver executed the receive primitive before message arrival, it would simply go back to its normal execution without the message, and the sender would remain blocked for ever.



# More about send

- ▶ The nonblocking send is more natural for many concurrent programming tasks.
- ▶ One potential danger of the nonblocking send is that an error could lead to a situation in which a process repeatedly generates messages, which could consume system resources, including processor time and buffer space.
- ▶ The nonblocking send places the burden on the programmer to determine that a message has been received.



# More about receive

- ▶ The blocking receive appears to be more natural for many concurrent programming tasks.
- ▶ If a message is lost, or if a process fails before it sends an anticipated message, a receiving process could be blocked indefinitely.
- ▶ We can allow a process to test whether a message is waiting before issuing a receive.
- ▶ We can allow a process to specify more than one source in a receive primitive.





# Lessons learned

- ▶ We explored the implementation of counting semaphores.
- ▶ We were introduced to one of the classical problems of synchronization, the producer / consumer problem and its versions.
- ▶ We implemented code for the producer / consumer problem using binary and counting semaphores.
- ▶ We were introduced to the concept of messages.



# Things to do



Read the book.



Write your questions on a piece of paper and keep it in front of you during the online session.



If you have suggestions relevant to content or quality of this lecture, email them to me.

